The Unicon Preprocessor

Jafar Al-Gharaibeh

Unicon Technical Report: 24

April 2026

Abstract

The Unicon compiler runs a line-oriented source preprocessor before lexical analysis. The implementation, uni/unicon/preproce.icn, comes from Bob Alexander's Jcon preprocessor. It has been extended with triple-quoted multiline strings, function-like $define macros, and built-in assert / assert_not. This report describes those extensions, the architecture and directives they sit on, how #line and includes interact with them, and how to add further facilities. Keywords: Unicon, preprocessor, triple-quoted strings, function-like macros, assertions, technical report.

Unicon Project
http://unicon.org
University of Idaho
Department of Computer Science
Moscow, ID, 83844, USA

1 1. Introduction

This report describes the Unicon source preprocessor: how a file plus includes becomes the token stream the compiler scans, what the `$` directives do, and the three extensions added in `uni/unicon/preproce.icn` – triple-quoted strings, function-like macros, and built-in `assert` / `assert_not`. It is formatted as a Unicon Technical Report [1]. The language-facing command list lives in *Programming with Unicon* (language reference, "Preprocessor") and UTR #8 [2]; that text still states that `$define` does not take arguments. Automated tests live under `tests/unicon/` (`triple_strings.icn`, `macros*.icn`, `assert_*.icn`). `unicon -E` writes preprocessed source.

The preprocessor is not optional. Every compilation runs it.

2 2. Motivation and prior art

The implementation is Bob Alexander's Jcon preprocessor, brought into Unicon with small edits [3]. It is line-oriented: it recognizes directives and identifiers, respects string and comment boundaries, and performs textual substitution. It is not a parser.

The Jcon baseline provided object-like `$define`, `$include`, `$ifdef` / `$ifndef` / `$else` / `$endif`, `$error`, `$line` / `#line`, and predefined symbols from `&features`. The implementation-book section "The Unicon Preprocessor" [4] documents that baseline (generator interface, include and `$ifdef` stacks, quoted-string skipping, underscore continuation). It still says macros have no parameters. The language reference says the same [2].

Three gaps in that baseline are filled here:

Multiline string literals. Unicon string literals are one physical line unless continued with a trailing `_`. A triple-quoted form lets a literal span lines, including text that looks like comments or `$` directives, without changing the lexer grammar.

Parameterized macros. Object-like `$define` cannot abstract repeated call-shaped text. Function-like `$define` adds a parameter list and a replacement body, with C-style \ continuation on the definition and call arguments that may span lines.

Assertions. Debug and test builds need a check that disappears entirely in release. Built-in `assert` / `assert_not` expand to runtime tests under `__debug__` / `__test__`, and to nothing otherwise.

3 3. Architecture

Canonical source is `uni/unicon/preproce.icn`. `uni/parser/preproce.icn` is a shorter, separate copy used by the standalone parser toolkit; it does not include the extensions in this report.

3.1 3.1 Public procedures

Procedure	Role
`preprocessor(fname, predefined_syms)`	Main entry. Initializes state, loops over input, emit
`preproc(dummy, args)`	Wrapper: `suspend preprocessor(args[1], pred`
`predefs()`	Table of predefined symbols (`_UNIX`, `__DATE__`,

`preproc_new()` initializes; `preproc_done()` clears large tables so they are not retained across compilations. `preproc_install_builtin_macros()` runs from `preproc_new()`. If `__test__` is enabled there, `__debug__` is defaulted.

3.2 3.2 State

`record preproc_fmacro(params, body)` holds a function-like macro: parameter names in order and replacement text. Object-like `$define` symbols live only in `preproc_sym_table`.

Global	Purpose
`preproc_sym_table`	Object-like macros:
`preproc_fun_table`	Function-like macros
`preproc_builtin_fun_table`	Names expanded spe
`preproc_if_stack`, `preproc_if_state`	Conditional compila
`preproc_file_stack`, `preproc_file`, `preproc_filename`, `preproc_line`	Include stack and cu
`preproc_include_set`, `preproc_include_name`	Included paths; circu
`preproc_print_line`, `preproc_print_filename`	Last emitted positio
`preproc_ml_anchor`	First physical line of
`preproc_nest_level`	`$if` depth at include
`preproc_word_chars`	Identifier characters:
`preproc_command`	Current directive na
`preproc_err_count`	Errors reported.

Global	Purpose
`preproc_dollar_or_pound`	Whether the directiv
`preproc_fmacro_parse_error`	Blocks object-like fa
`preproc_assert_uid`	Suffix for unique tem

3.3 3.3 Driver

`preprocessor()` is a generator. The translator drains it into `yyin`:

```
every yyin ||:= preprocessor(fname, uni_predefs) do
   yyin ||:= "                          n"
```

in `uni/unicon/unicon.icn`. `uni_predefs` is `predefs()` plus `-D` options. After the generator finishes, a non-zero `preproc_err_count` or a non-empty `parsingErrors` aborts before `yyparse()`.

Each line from `preproc_read()` is scanned under `line ? { ... }`. After leading `preproc_space()`:

- `#line` (optional spaces, GCC form) goes to `preproc_scan_directive()`.
- `$` followed by a character in `nonpunctuation` (letters, digits, space, tab, form feed, carriage return) goes to `preproc_scan_directive()`.
- Otherwise `&pos := 1` and the line goes to `preproc_scan_text()`.

Unicon directives are `$`-prefixed. A `#` that is not `#line` is not a directive. With macros loaded, `preproc_scan_text()` treats `#` outside strings as a comment to end of line. Do not rely on C-style `#define` at column 0.

There is no second pass over the same line.

3.4 3.4 Conditional compilation

`/preproc_if_state` is Unicon's null test. After `$ifdef`, the true branch sets `preproc_if_state` to null (emit); the false branch sets `"false"` or `"off"` (skip). So `/preproc_if_state` means the current region is active (or there is no `$ifdef`). `\preproc_if_state` is the nonnull test.

`$define`, `$undef`, `$include`, and text expansion run only when `/preproc_if_state` succeeds. `preproc_scan_text()` returns without emitting in skipped regions

– no triple-string rewrite, no macro expansion. `$ifdef` / `$ifndef` / `$else` / `$endif` still run in skipped regions so nested conditionals stay balanced. A bare `if` token inside skipped text is mapped to `$if`, which pushes state and forces `"off"`.

3.5 3.5 Text scan order

When the region is active, `preproc_scan_text()`:

1. Rewrites `"""` ... `"""` via `preproc_rewrite_triple_strings()` before identifier scans, so the body is never treated as directives or macros.
2. If either symbol table is non-empty, scans for `#`, strings, and identifiers. Otherwise it emits the line after `preproc_sync_lines()`.
3. Looks up function-like macros before object-like. A name in `preproc_fun_table` is a call only if `preproc_expand_fmacro_call()` sees `(` after optional space; otherwise `&pos` is restored and the name may still match an object-like macro.
4. Rescans replacements through `preproc_scan_text(done_set)`. `done_set` is `&null`, a string, or a set of names on the current expansion chain; it blocks infinite recursion.

String literals `"..."` / `'...'` are skipped with escape and trailing-`_` continuation (`preproc_read()`, adjusting `preproc_print_line`).

4 4. Directives

`preproc_scan_directive()` reads the command with `preproc_word()`. Unknown names in an active region call `preproc_error("unknown preprocessor directive")`.

Command	Behavior
`define`	Function-like `name(params...) body` via
`undef`	Removes the name from both tables and from the builtin
`ifdef` / `ifndef`	Pushes `preproc_if_state`; sets state from
`$if`	Placeholder for `if` tokens inside excluded code:
`else` / `endif`	Pop, toggle, or close the branch; error on mismatch.

Command	Behavior
`include`	Quoted or bare filename via `preproc_qword()`; circular
`line`	Sets `preproc_filename` / `preproc_line` from a
`error`	User error; message to end of line or `#`.
`ITRACE`	Sets `&trace` from an integer.
`C`	Accumulates lines until `$Cend`; passes the text to

5 5. Triple-quoted multiline strings

Delimiters `"""` ... `"""` mark a string whose body may span physical lines. The preprocessor collects raw text – including lines that look like comments or `$` directives – and rewrites the region to an ordinary quoted literal. The lexer grammar in `uni/parser` is unchanged.

```
procedure main()
   local x
   x := """line one
line two"""
   write(x)
end
```

Output:

```
line one
line two
```

The opener may sit on the same line as code or alone on a following line. Indentation and newlines in the body are kept. Multiline bodies do not shift `&line` for surrounding statements (`triple_strings.icn`).

```
procedure main()
   local x
   x := """
$ifdef __debug__
inside triple
$endif
"""
```

```
   write("[", x, "]")
   x := """escaped triple:       "       "       " inside"""
   write(x)
   x := "contains         "         "         " literally"
   write(x)
   x := """ends-with-backslash                 """
   write("[", x, "]")
end
```

Output:

```
[
$ifdef __debug__
inside triple
$endif
]
escaped triple: """ inside
contains """ literally
[ends-with-backslash                        ]
```

Inside "..." or '...', a """ sequence is copied through and does not start this mode. To put three double quotes in a triple-quoted body, write \"\"\" (backslash before each quote). The four-character form \""" is not an escape, so a body may end with a single \.

`$ifdef` / `$endif` and # comments inside the body are string content. Continuation lines pulled for an open triple are never scanned as directives.

`preproc_rewrite_triple_strings()` runs at the start of `preproc_scan_text()`, only in an active region, and only if the current line contains """. Outside quotes it still honors # as a comment to end of that physical line. After an opener, it accumulates a body until the closer; if the line ends first, it appends a newline, `preproc_read()`s the next line, and continues. An unclosed delimiter at EOF is `unterminated triple-quoted string`.

The body is turned into one ordinary string with `preproc_quote_string()`: named Unicon escapes for specials (including DEL as `\d`), backslash and quote escaped, other C0 bytes as three-digit octal via `preproc_octal_escape()`. `triple_strings.icn` checks that a DEL byte in a triple string survives compilation (`ord` 127).

Tests: `tests/unicon/triple_strings.icn`, `tests/unicon/stand/triple_strings.std`.

6 6. Function-like macros

Function-like macros extend object-like `$define` with a parameter list and a replacement body. Nested calls, zero arguments, and commas inside quoted arguments are supported (`macros.icn`):

```
$define SUM(a,b) ((a) + (b))
$define TWICE(x) SUM(x,x)
$define ZERO() 0
procedure main()
   write(SUM(2, 3))
   write(TWICE(5))
   write(ZERO())
end
```

Output:

```
5
10
0
```

Quoted commas and parentheses do not split arguments. A call may continue on the next line without \:

```
$define ID(x) x
$define WRAP3(a,b,c) ((a) || "|" || (b) || "|" || (c))
$define SUM(a,b) ((a) + (b))
procedure main()
   write(ID("a,b"))
   write(WRAP3("a,b", "c(d)", "e"))
   write(SUM(1,
      2))
end
```

Output:

```
a,b
a,b|c(d)|e
3
```

On `$define`, `(` must follow the name immediately. `$define SUM (a,b) ...` is object-like: the value starts at the space. At a call site, space before `(` is allowed (`preproc_opt_space()`), so a body may write `SUM (x,y)`.

Zero parameters are legal (`ZERO()`). Whitespace-only between `(` and `)` is zero arguments (`ZERO( )`), not one empty argument.

The replacement may continue on following lines if each continued line ends with \ as the very last character (no trim). `preproc_scan_define_value()` and `preproc_define_value_next_line()` splice those lines: the \ and newline are dropped; other spaces are kept. A \ with trailing spaces is not a continuation. Leading space is skipped only before the first line of the body. The same \ rule applies to object-like `$define` values (`macros_define_continuation.icn`). EOF after a splice is `unterminated $define (line continuation)`.

```
$define GREETING "hello " ||
"there"
$define SUMCONT(a,b) ((a) +
(b))
procedure main()
   write(GREETING)
   write(SUMCONT(2, 3))
end
```

Output:

```
hello there
5
```

A call `NAME(arg1, ...)` may span physical lines without \. `preproc_scan_macro_args()` pulls more lines via `preproc_macro_arg_next_line()` until the closing `)`. A backslash inside a call is ordinary argument text (`macros_call_backslash_literal.icn`), not a `$define` splice.

Arguments split on top-level commas, with parenthesis depth, and with `"..."` / `'...'` opaque (commas and parens inside quotes do not split). `preproc_subst_fmacro()` replaces whole identifiers in the body, not substrings, and does not substitute inside strings. The result is rescanned, so `TWICE` and chains such as `A -> E` expand in one pipeline. `done_set` blocks expanding the same name again on that chain.

Empty arguments (`SUM(1,)`) are `empty macro argument`. Duplicate parameter names are `duplicate parameter name in $define`. Arity must match (`wrong number of args in macro call to NAME`). Unclosed `(` is `unterminated macro ir` If `(` follows the name on `$define` but the parameter list is malformed, `preproc_fmacro_parse_error` is set so the line is not treated as a bad object-like `$define`.

Call continuation anchors `preproc_print_line` to the first physical line of the call (`preproc_ml_anchor`). `&line` inside the expansion is that line; the next source line after the call skips the continuation rows. `$define` \ splicing does not adjust `preproc_print_line`. `&line` in a multiline body is the invocation line, not each physical line of the definition (`macros_define_multiline_line.icn`).

```
$define ASLINE(x) x

procedure main()
   write(&line)
   write(ASLINE(&line
   ))
   write(&line)
end
```

Output:

```
4
5
7
```

(The call starts on line 5; line 6 is the continuation; the next statement is line 7.)

A function-like name is defined for `$ifdef`. `$undef` removes it. A name is either function-like or object-like, not both; `$define` updates one table and clears the other. Redefinition of a function-like name is an error unless `preproc_same_fmacro()` finds the same parameters and body.

```
$define SUM(a,b) ((a) + (b))
$ifdef SUM
$define HAS_SUM 1
$else
```

```
$define HAS_SUM 0
$endif
$undef SUM
$ifdef SUM
$define STILL 1
$else
$define STILL 0
$endif
procedure main()
   write(HAS_SUM)
   write(STILL)
end
```

Output:

```
1
0
```

`$undef assert` then `$define assert(...)` replaces the builtin; `macros.icn` does this for both `assert` and `assert_not`.

Tests: `tests/unicon/macros.icn`; `macros_multiline_line.icn`, `macros_define_multiline` `macros_define_continuation.icn`, `macros_call_backslash_literal.icn`; negatives under `tests/unicon/data/macros_bad_*.icn` and `macros_bad_define_eof*.icn`.

7 7. Built-in `assert` and `assert_not`

`preproc_install_builtin_macros()` registers `assert` and `assert_not` in `preproc_fun_table` as placeholder `preproc_fmacro` records and marks them in `preproc_builtin_fun_table`. Expansion is not `preproc_subst_fmacro()`: `preproc_expand_fmacro_call()` builds a Unicon fragment from the argument text. Arity is one or two: `assert(expr)` or `assert(expr, label)`. Any other count is `wrong number of args in macro call`.

Semantics follow Unicon success and failure, not Boolean truth: `assert(expr)` expects `expr` to succeed; `assert_not(expr)` expects `expr` to fail. `assert` is alternation (`expr | { ... }`); `assert_not` is `if (expr) then { ... }`.

Mode	Enable	On failure
Debugging	`$define __debug__` or `-D__debug__`	`runerr(219, payload)`. The process stops.
Testing	`$define __test__` or `-D__test__`	`write` of `[file:line] assertion failed`
Release	neither	Expands to nothing (`assert_strip.icn`).

If both `__test__` and `__debug__` are defined, the testing path (`fail`) wins. `preproc_new()` and a later `$define __test__` both default `__debug__` when it is absent.

These macros are standalone statements. Disabled expansion is empty, so expression positions such as `if assert(...)` are not supported.

The debug payload is the quoted expression text, then `char(30)` and the label if the label is non-empty (`AssertRunerrSep` in `errmsg.r`). `assert_not` prefixes `char(29)` so the traceback can print `assert_not(`. Error 219 is `assertion failed` (`data.r`). `errmsg.r` skips the ordinary `Run-time error 219` banner and the offending- value line; it prints `[file:line] assertion failed: ...`.

If the label expression fails, the assertion diagnostic still runs: the expansion does `lbl := ((label) | "")` (`assert_failing_label.icn`).

Each expansion increments `preproc_assert_uid` so generated temporaries (`__preproc_assert_N_lbl`) do not collide – icont has no block-local variables in expression-level `{...}` compounds. The fragment is rescanned so nested macros in the inserted code expand.

`$undef assert` then `$define assert(...)` replaces the builtin like any user function-like macro.

```
$define __debug__
procedure main()
   local x
   x := 1
   assert(x = 1)
   assert_not(x = 2)
   assert(2 === 1 + 1, "1 + 1 = 2")
   write("ok")
end
```

Output:

```
ok
```

A failing debug assert stops the process. The traceback names `assert` / `assert_not` rather than `runerr`:

```
$define __debug__
procedure main()
   assert_not(1 = 1)
   write("should not reach here")
end
```

Output:

```
[assert_ex.icn:3] assertion failed: 1 = 1
Traceback:
   main()
   assert_not(1 = 1) from line 3 in assert_ex.icn
```

Under `__test__`, failure writes a diagnostic and `fail`s, so later statements still run:

```
$define __test__
procedure main()
   assert(1 = 2, "keep going")
   write("after fail")
   assert(1 = 1)
   write("done")
end
```

Output:

```
[assert_ex.icn:3] assertion failed (1 = 2) keep going
after fail
done
```

With neither symbol, the calls vanish. The asserts below would fail at run time if they were still present, so reaching `write()` is the proof they were stripped:

```
procedure main()
   local x
   x := 1
   assert(x = 2)
   assert_not(x = 1)
   write("strip-ok")
end
```

Output:

```
strip-ok
```

Tests: `assert_debugging.icn` (passing cases, then a failing assert five frames down), `assert_testing.icn`, `assert_strip.icn`, `assert_failing_label.icn`, `assert_not_failing.icn`, matching `stand/*.std`. Builtin override is in `macros.icn`.

Concern	Primary routines (`uni/unicon/preproce.icn`)
Triple-quoted strings	`preproc_rewrite_triple_strings()`,
Function-like macros	`record preproc_fmacro`,
Assertions	`preproc_install_builtin_macros()`,

8 8. Line numbers, errors, and includes

`preproc_sync_lines()` emits `#line N "file"` when the filename changes or the gap is large; for small gaps it may emit blank lines. `preproc_scan_text()` can `suspend` more than once per logical line. Features that insert, delete, or merge lines must keep `preproc_line` and `preproc_print_line` consistent with that.

`preproc_error(msg)` builds `File ...; Line N # $directive: ...`, increments `preproc_err_count`, and pushes a `ParseError` onto `parsingErrors`.

`preproc_read()` increments `preproc_line` and returns the next line from `read()` or from a list "file." On EOF it checks `preproc_if_stack` depth against `preproc_nest_level`, closes the file, pops `preproc_file_stack`, and removes the path from `preproc_include_set`. String continuation,

triple-quoted bodies, and macro arguments that need another physical line all call `preproc_read()` again.

`preproc_quote_string(s)` escapes arbitrary text for a Unicon string literal. `preproc_istring()` / `preproc_istring_radix()` decode escapes in quoted filenames.

9 9. Extending the preprocessor

All of the following is in `uni/unicon/preproce.icn` unless noted. Coordinate any change with `uni/parser/preproce.icn` if that copy is still shipped; it does not currently match.

9.1 9.1 Kind and touchpoints

Kind	Touchpoints
New or changed `$` directive	`preproc_scan_directive()`: add a `case` label; set
Object-like macro behavior	`preproc_sym_table`, substitution in
Function-like macro behavior	`preproc_fun_table`,
Built-in special function macro	`preproc_install_builtin_macros()`,
Source syntax rewritten to plain Unicon	A phase in `preproc_scan_text()`. Must respect
New predefined symbol	`predefs()`. Symbols that depend on compiler options

9.2 9.2 Guards and line accounting

Directives that define, undefine, or include use `if /preproc_if_state then` so they do not run when code is stripped. Directives that only keep parser state in skipped regions follow `ifdef` / `$if` / `else` / `endif`, not that guard.

If the feature reads extra lines or changes output length, check `preproc_line`, `preproc_print_line`, and `preproc_sync_lines()`. `unicon -E` shows the emitted `#line` stream. Multiple `suspend`s per input line must leave `preproc_print_line` consistent at the end of `preproc_scan_text()`.

9.3 9.3 Errors and tests

Use `preproc_error(msg)` after `preproc_command` is set so the message includes `$directive:`. The compiler exit path is `preproc_err_count` and

`parsingErrors` in `unicon.icn`.

Add a driver under `tests/unicon/` and expected output under `stand/`. Error cases belong in `tests/unicon/data/`. Update sections 5–7 when user-visible semantics change.

9.4 9.4 New $ directive

After `preproc_command := preproc_word()`, the `case preproc_command of` dispatch matches the word after `$`. A new label:

```
"mydir": {
   if /preproc_if_state then {
      # parse remainder of line; on failure:
      preproc_error("optional detail")
   }
}
```

Unknown names still hit `default` and `preproc_error("unknown preprocessor directive")` when active.

10 10. Related documents

Document	Content
UTR #15 [1]	How to write and submit a Unicon Technical Report.
UTR #8 [2]	Language reference preprocessor commands
Implementation book [4]	`doc/ib/p3-unicon.tex`, "The Unicon Preprocessor":

Other `doc/ib/` preprocessor mentions (Idol, Ratfor, rtt) are different tools, not `preproce.icn`.

11 References

References

[1] Clinton Jeffery. "How to Write a Unicon Technical Report". doc/utr/utr15.tex. 2013.

[2] Clinton Jeffery. "Unicon Language Reference". doc/unicon/utr8.html.

[3] Bob Alexander. "Preprocessor for Jcon". Basis for Unicon's preproce.icn.

[4] Clinton Jeffery. "The Unicon Compiler". Course notes, doc/ib/p3-unicon.tex, section "The Unicon Preprocessor".

[5] The libssh Project. "libssh – The SSH Library". https://www.libssh.org/. 2026.

[6] Clinton Jeffery. "The Unicon Messaging Facilities". doc/utr/utr13.odt.

[7] The OpenSSL Project. "OpenSSL – Cryptography and SSL/TLS Toolkit". https://www.openssl.org/. 2026.

[8] Jafar Al-Gharaibeh. "Native SSH and SFTP Support in Unicon". doc/utr/utr26.rst. 2026.

[9] H. Krawczyk and M. Bellare and R. Canetti. "HMAC: Keyed-Hashing for Message Authentication". RFC 2104. 1997.